

Corrigé – TD Algorithmique

Apprendre et Maîtriser les Bases

avec AlgoBox

Cours : Introduction à l'algorithmique

Outil : AlgoBox (logiciel gratuit)

Niveau : Débutant – IUT / BTS / Licence

Ce document contient les corrections détaillées de tous les exercices du TD.

- Algorithmes complets en pseudo-code AlgoBox
- Explications pas à pas
- Traces d'exécution et résultats
- Remarques pédagogiques

Table des matières

1	Introduction à AlgoBox	2
2	Variables et affectation	3
3	Entrées et sorties	7
4	Structures conditionnelles	8
5	Structures itératives (boucles)	12
6	Tableaux	17
7	Fonctions	22
8	Exercices de synthèse	25

1 Introduction à AlgoBox

Correction 1.1 – Découverte de l'interface

1. **Zone de déclaration des variables :** En haut de la fenêtre, on trouve la section **VARIABLES** où l'on déclare le nom et le type de chaque variable.
2. **Zone d'écriture de l'algorithme :** La zone centrale (grand cadre) permet de saisir les instructions de l'algorithme entre **DEBUT_ALGORITHME** et **FIN_ALGORITHME**.
3. **Boutons :**
 - *Nouveau* : crée un nouvel algorithme vide.
 - *Ouvrir* : charge un algorithme sauvegardé.
 - *Tester* : exécute l'algorithme en continu.
 - *Pas à pas* : exécute l'algorithme instruction par instruction.
4. L'algorithme saisi affiche une boîte de dialogue demandant de saisir un prénom, puis affiche "Bonjour <prénom> !".
5. En mode *Pas à pas*, on observe chaque instruction une par une : d'abord la lecture du prénom, puis l'affichage séquentiel de chaque partie du message.

Correction 1.2 – Premier algorithme : calcul simple

```
VARIABLES
a EST_DU_TYPE NOMBRE
b EST_DU_TYPE NOMBRE
somme EST_DU_TYPE NOMBRE
diff EST_DU_TYPE NOMBRE
prod EST_DU_TYPE NOMBRE
quot EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE a
  LIRE b
  somme PREND_LA_VALEUR a + b
  diff PREND_LA_VALEUR a - b
  prod PREND_LA_VALEUR a * b
  AFFICHER "Somme = "
  AFFICHER somme
  AFFICHER " - Difference = "
  AFFICHER diff
  AFFICHER " - Produit = "
  AFFICHER prod
  SI (b != 0) ALORS
    DEBUT_SI
      quot PREND_LA_VALEUR a / b
      AFFICHER " - Quotient = "
      AFFICHER quot
    FIN_SI
  SINON
    DEBUT_SINON
      AFFICHER " - Division par zero impossible"
    FIN_SINON
FIN_ALGORITHME
```

Remarque

Si $b = 0$, la division par zéro provoque une erreur. On ajoute donc un test conditionnel pour protéger cette opération.

Correction 1.3 – Types de variables

1. L'âge d'une personne → NOMBRE (entier, mais AlgoBox n'a pas de type entier distinct).
2. Le nom d'une ville → CHAINE (texte).
3. Le fait qu'un nombre soit pair ou impair → BOOLEEN (vrai/faux).
4. La température extérieure → NOMBRE (peut être décimale, ex. : 23,5).
5. Un numéro de téléphone → CHAINE (pour conserver les zéros en tête et les espaces).
6. La réponse à « $a > b$? » → BOOLEEN (résultat d'une comparaison).
7. La moyenne générale → NOMBRE (valeur décimale).
8. Le code postal → CHAINE (certains codes commencent par 0, ex. : 01234).

Correction 1.4 – Affectation et trace

1. Tableau de trace complété :

Instruction	a	b	c
$a \leftarrow 3$	3	–	–
$b \leftarrow 5$	3	5	–
$c \leftarrow a+b$	3	5	8
$a \leftarrow c-a$	5	5	8
$b \leftarrow a+2$	5	7	8

2. Valeurs affichées : $a = 5$, $b = 7$, $c = 8$
3. Vérifiez dans AlgoBox : les valeurs affichées confirment la trace.

2 Variables et affectation**Correction 2.1 – Échange de deux variables**

1. Avec variable temporaire :

```

VARIABLES
  x EST_DU_TYPE NOMBRE
  y EST_DU_TYPE NOMBRE
  temp EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  x PREND_LA_VALEUR 7
  y PREND_LA_VALEUR 12
  temp PREND_LA_VALEUR x
  x PREND_LA_VALEUR y
  y PREND_LA_VALEUR temp
  AFFICHER "x = "
  AFFICHER x
  AFFICHER " y = "
```

```

AFFICHER y
FIN_ALGORITHME

```

Résultat : $x = 12$, $y = 7$ ✓

2. Sans variable temporaire (méthode arithmétique) :

```

x PREND_LA_VALEUR x + y // x = 19
y PREND_LA_VALEUR x - y // y = 19-12 = 7
x PREND_LA_VALEUR x - y // x = 19-7 = 12

```

Remarque

Cette méthode fonctionne mais peut poser des problèmes de précision avec les nombres flottants (dépassement de capacité possible pour de grandes valeurs). La méthode avec variable temporaire est plus sûre.

Correction 2.2 – Affectations enchaînées

Séquence 1 :

```

a PREND_LA_VALEUR 2 // a=2
b PREND_LA_VALEUR a+3 // b=5
a PREND_LA_VALEUR a+b // a=2+5=7
c PREND_LA_VALEUR a*b // c=7*5=35

```

Résultat : $a = 7$, $b = 5$, $c = 35$

Séquence 2 :

```

a PREND_LA_VALEUR 10 // a=10
b PREND_LA_VALEUR a-4 // b=6
a PREND_LA_VALEUR b // a=6
c PREND_LA_VALEUR a+b+1 // c=6+6+1=13

```

Résultat : $a = 6$, $b = 6$, $c = 13$

Séquence 3 :

```

a PREND_LA_VALEUR 1 // a=1
b PREND_LA_VALEUR 2 // b=2
c PREND_LA_VALEUR 3 // c=3
a PREND_LA_VALEUR b // a=2
b PREND_LA_VALEUR c // b=3
c PREND_LA_VALEUR a // c=2

```

Résultat : $a = 2$, $b = 3$, $c = 2$

Correction 2.3 – Calculs mathématiques

```

VARIABLES
x EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
LIRE x
AFFICHER "x^2 = "
AFFICHER pow(x,2)
AFFICHER " - x^3 = "
AFFICHER pow(x,3)
SI (x >= 0) ALORS

```

```

    DEBUT_SI
        AFFICHER " - sqrt(x) = "
        AFFICHER sqrt(x)
    FIN_SI
SINON
    DEBUT_SINON
        AFFICHER " - sqrt(x) : impossible"
    FIN_SINON
SI (x != 0) ALORS
    DEBUT_SI
        AFFICHER " - 1/x = "
        AFFICHER 1/x
    FIN_SI
SINON
    DEBUT_SINON
        AFFICHER " - 1/x : division par zero"
    FIN_SINON
FIN_ALGORITHME

```

Remarque

On protège les calculs de $\sqrt{x}$ (domaine $x \geq 0$) et de $\frac{1}{x}$ (domaine $x \neq 0$) avec des tests conditionnels.

Correction 2.4 – Conversion d'unités

Celsius → Fahrenheit :

```

VARIABLES
    C EST_DU_TYPE NOMBRE
    F EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE C
    F PREND_LA_VALEUR (9/5)*C + 32
    AFFICHER C
    AFFICHER " degC = "
    AFFICHER F
    AFFICHER " degF"
FIN_ALGORITHME

```

Fahrenheit → Celsius :

```

VARIABLES
    F EST_DU_TYPE NOMBRE
    C EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE F
    C PREND_LA_VALEUR (5/9)*(F - 32)
    AFFICHER F
    AFFICHER " degF = "
    AFFICHER C
    AFFICHER " degC"
FIN_ALGORITHME

```

Correction 2.5 – Aires et périmètres**1. Rectangle :**

```
VARIABLES
  L EST_DU_TYPE NOMBRE
  l EST_DU_TYPE NOMBRE
  aire EST_DU_TYPE NOMBRE
  perim EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE L
  LIRE l
  aire PREND_LA_VALEUR L * l
  perim PREND_LA_VALEUR 2*(L + l)
  AFFICHER "Aire = "
  AFFICHER aire
  AFFICHER " - Perimetre = "
  AFFICHER perim
FIN_ALGORITHME
```

2. Cercle :

```
VARIABLES
  r EST_DU_TYPE NOMBRE
  aire EST_DU_TYPE NOMBRE
  circ EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE r
  aire PREND_LA_VALEUR PI * pow(r,2)
  circ PREND_LA_VALEUR 2 * PI * r
  AFFICHER "Aire = "
  AFFICHER aire
  AFFICHER " - Circonference = "
  AFFICHER circ
FIN_ALGORITHME
```

3. Triangle :

```
VARIABLES
  base EST_DU_TYPE NOMBRE
  h EST_DU_TYPE NOMBRE
  aire EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE base
  LIRE h
  aire PREND_LA_VALEUR (base * h) / 2
  AFFICHER "Aire = "
  AFFICHER aire
FIN_ALGORITHME
```

3 Entrées et sorties

Correction 3.1 – Affichage formaté

```
VARIABLES
  prenom EST_DU_TYPE CHAINE
  age EST_DU_TYPE NOMBRE
  ville EST_DU_TYPE CHAINE
DEBUT_ALGORITHME
  LIRE prenom
  LIRE age
  LIRE ville
  AFFICHER "Bonjour, je m'appelle "
  AFFICHER prenom
  AFFICHER ", j'ai "
  AFFICHER age
  AFFICHER " ans et j'habite "
  AFFICHER ville
  AFFICHER "."
FIN_ALGORITHME
```

Remarque

On utilise AFFICHER (sans étoile) pour chaque élément afin que tout s'affiche sur une seule ligne. AFFICHER* provoque un retour à la ligne.

Correction 3.2 – Calcul du prix TTC

```
VARIABLES
  HT EST_DU_TYPE NOMBRE
  TVA EST_DU_TYPE NOMBRE
  TTC EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE HT
  LIRE TVA
  TTC PREND_LA_VALEUR HT * (1 + TVA/100)
  AFFICHER "Prix HT : "
  AFFICHER HT
  AFFICHER " euros - TVA : "
  AFFICHER TVA
  AFFICHER "% - Prix TTC : "
  AFFICHER TTC
  AFFICHER " euros"
FIN_ALGORITHME
```

Correction 3.3 – Moyenne de notes

```
VARIABLES
  n1 EST_DU_TYPE NOMBRE
  n2 EST_DU_TYPE NOMBRE
  n3 EST_DU_TYPE NOMBRE
  moy EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n1
```

```
LIRE n2
LIRE n3
moy PREND_LA_VALEUR (n1+n2+n3)/3
AFFICHER "Note 1 = "
AFFICHER n1
AFFICHER " - Note 2 = "
AFFICHER n2
AFFICHER " - Note 3 = "
AFFICHER n3
AFFICHER " - Moyenne = "
AFFICHER moy
SI (moy >= 10) ALORS
    DEBUT_SI
        AFFICHER " (passable)"
    FIN_SI
SINON
    DEBUT_SINON
        AFFICHER " (insuffisant)"
    FIN_SINON
FIN_ALGORITHME
```

Correction 3.4 – Simulation de sortie AlgoBox

Avec les saisies 5, 3, 8 :

- $a = 5, b = 3, c = 8$
- $s = 5 + 3 + 8 = 16$
- $m = 16/3 \approx 5,333...$

Affichage dans AlgoBox :

```
Somme = 16
Moyenne = 5.333333333333333
```

Remarque

AFFICHER* provoque un retour à la ligne après chaque affichage, d'où les deux lignes distinctes.

4 Structures conditionnelles

Correction 4.1 – Pair ou impair

```
VARIABLES
    n EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    SI (n MOD 2 == 0) ALORS
        DEBUT_SI
            AFFICHER n
            AFFICHER " est pair"
        FIN_SI
    SINON
        DEBUT_SINON
```

```
AFFICHER n
AFFICHER " est impair"
FIN_SINON
FIN_ALGORITHME
```

Correction 4.2 – Maximum de deux nombres

```
VARIABLES
a EST_DU_TYPE NOMBRE
b EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
LIRE a
LIRE b
SI (a > b) ALORS
    DEBUT_SI
        AFFICHER "Le maximum est : "
        AFFICHER a
    FIN_SI
SINON
    DEBUT_SINON
        AFFICHER "Le maximum est : "
        AFFICHER b
    FIN_SINON
FIN_ALGORITHME
```

Correction 4.3 – Maximum de trois nombres

```
VARIABLES
a EST_DU_TYPE NOMBRE
b EST_DU_TYPE NOMBRE
c EST_DU_TYPE NOMBRE
max EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
LIRE a
LIRE b
LIRE c
max PREND_LA_VALEUR a
SI (b > max) ALORS
    DEBUT_SI
        max PREND_LA_VALEUR b
    FIN_SI
SI (c > max) ALORS
    DEBUT_SI
        max PREND_LA_VALEUR c
    FIN_SI
AFFICHER "Le maximum est : "
AFFICHER max
FIN_ALGORITHME
```

Remarque

On initialise max avec a, puis on compare successivement avec b et c. Cette approche se généralise facilement à n nombres avec une boucle.

Correction 4.4 – Année bissextile

```

VARIABLES
  annee EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE annee
  SI ((annee MOD 4 == 0 ET annee MOD 100 != 0)
      OU annee MOD 400 == 0) ALORS
    DEBUT_SI
      AFFICHER annee
      AFFICHER " est bissextile"
    FIN_SI
  SINON
    DEBUT_SINON
      AFFICHER annee
      AFFICHER " n'est pas bissextile"
    FIN_SINON
FIN_ALGORITHME

```

Exemples :

- 2024 : divisible par 4 et non par 100 → **bissextile**
- 2100 : divisible par 100 et non par 400 → **non bissextile**
- 2000 : divisible par 400 → **bissextile**

Correction 4.5 – Calcul de l'impôt simplifié

```

VARIABLES
  R EST_DU_TYPE NOMBRE
  impot EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE R
  SI (R <= 10084) ALORS
    DEBUT_SI
      impot PREND_LA_VALEUR 0
    FIN_SI
  SINON
    DEBUT_SINON
      SI (R <= 25710) ALORS
        DEBUT_SI
          impot PREND_LA_VALEUR R * 0.11
        FIN_SI
      SINON
        DEBUT_SINON
          SI (R <= 73516) ALORS
            DEBUT_SI
              impot PREND_LA_VALEUR R * 0.30
            FIN_SI
          SINON
            DEBUT_SINON
              impot PREND_LA_VALEUR R * 0.41
            FIN_SINON
          FIN_SINON
        FIN_SINON
      FIN_SINON
    FIN_SINON
  AFFICHER "Impot = "
  AFFICHER impot

```

```
AFFICHER " euros"
FIN_ALGORITHME
```

Remarque

Ce barème est simplifié : en réalité, l'impôt français est progressif (chaque tranche est taxée à son propre taux). L'exercice utilise un taux unique appliqué au revenu total.

Correction 4.6 – Mention du baccalauréat

```
VARIABLES
  m EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE m
  SI (m < 8) ALORS
    DEBUT_SI
      AFFICHER "Recale"
    FIN_SI
  SINON
    DEBUT_SINON
      SI (m < 10) ALORS
        DEBUT_SI
          AFFICHER "Rattrapage"
        FIN_SI
      SINON
        DEBUT_SINON
          SI (m < 12) ALORS
            DEBUT_SI
              AFFICHER "Passable"
            FIN_SI
          SINON
            DEBUT_SINON
              SI (m < 14) ALORS
                DEBUT_SI
                  AFFICHER "Assez bien"
                FIN_SI
              SINON
                DEBUT_SINON
                  SI (m < 16) ALORS
                    DEBUT_SI
                      AFFICHER "Bien"
                    FIN_SI
                  SINON
                    DEBUT_SINON
                      AFFICHER "Tres bien"
                    FIN_SINON
                  FIN_SINON
                FIN_SINON
              FIN_SINON
            FIN_SINON
          FIN_SINON
        FIN_SINON
      FIN_SINON
    FIN_SINON
  FIN_SINON
FIN_ALGORITHME
```

Correction 4.7 – Opérations sur les chaînes

```

VARIABLES
  mot EST_DU_TYPE CHAINE
  prem EST_DU_TYPE CHAINE
DEBUT_ALGORITHME
  LIRE mot
  prem PREND_LA_VALEUR mot.substring(0,1)
  SI (prem == "a" OU prem == "e" OU prem == "i"
      OU prem == "o" OU prem == "u" OU prem == "y") ALORS
    DEBUT_SI
      AFFICHER mot
      AFFICHER " commence par une voyelle"
    FIN_SI
  SINON
    DEBUT_SINON
      AFFICHER mot
      AFFICHER " commence par une consonne"
    FIN_SINON
  FIN_ALGORITHME

```

Remarque

En AlgoBox, la comparaison de chaînes se fait avec ==. La fonction `substring(début, fin)` extrait une sous-chaîne (l'indice de fin est exclusif).

5 Structures itératives (boucles)

Correction 5.1 – Somme des n premiers entiers

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  S EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  S PREND_LA_VALEUR 0
  POUR i ALLANT_DE 1 A n
    DEBUT_POUR
      S PREND_LA_VALEUR S + i
    FIN_POUR
  AFFICHER "Somme = "
  AFFICHER S
  AFFICHER " - Verification : n(n+1)/2 = "
  AFFICHER n*(n+1)/2
FIN_ALGORITHME

```

Vérification : pour $n = 10$, $S = 55$ et $\frac{10 \times 11}{2} = 55 \checkmark$

Correction 5.2 – Table de multiplication

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE

```

```

    res EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    POUR i ALLANT_DE 1 A 10
        DEBUT_POUR
            res PREND_LA_VALEUR i * n
            AFFICHER i
            AFFICHER " x "
            AFFICHER n
            AFFICHER " = "
            AFFICHER* res
        FIN_POUR
    FIN_ALGORITHME

```

Correction 5.3 – Factorielle

Version avec boucle POUR :

```

VARIABLES
    n EST_DU_TYPE NOMBRE
    fact EST_DU_TYPE NOMBRE
    i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    fact PREND_LA_VALEUR 1
    POUR i ALLANT_DE 1 A n
        DEBUT_POUR
            fact PREND_LA_VALEUR fact * i
        FIN_POUR
    AFFICHER n
    AFFICHER "! = "
    AFFICHER fact
FIN_ALGORITHME

```

Version avec boucle TANT_QUE :

```

VARIABLES
    n EST_DU_TYPE NOMBRE
    fact EST_DU_TYPE NOMBRE
    i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    fact PREND_LA_VALEUR 1
    i PREND_LA_VALEUR 1
    TANT_QUE (i <= n) FAIRE
        DEBUT_TANT_QUE
            fact PREND_LA_VALEUR fact * i
            i PREND_LA_VALEUR i + 1
        FIN_TANT_QUE
    AFFICHER n
    AFFICHER "! = "
    AFFICHER fact
FIN_ALGORITHME

```

Remarque

Pour $n = 0$, la boucle ne s'exécute pas et **fact** reste 1, ce qui est correct car $0! = 1$.

Correction 5.4 – Puissance

```

VARIABLES
  x EST_DU_TYPE NOMBRE
  n EST_DU_TYPE NOMBRE
  resultat EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE x
  LIRE n
  resultat PREND_LA_VALEUR 1
  POUR i ALLANT_DE 1 A n
    DEBUT_POUR
      resultat PREND_LA_VALEUR resultat * x
    FIN_POUR
  AFFICHER x
  AFFICHER "^"
  AFFICHER n
  AFFICHER " = "
  AFFICHER resultat
FIN_ALGORITHME

```

Remarque

Pour $n = 0$, la boucle ne s'exécute pas et le résultat vaut 1, ce qui est correct : $x^0 = 1$ pour tout $x \neq 0$.

Correction 5.5 – Deviner un nombre

```

VARIABLES
  secret EST_DU_TYPE NOMBRE
  proposition EST_DU_TYPE NOMBRE
  essais EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  secret PREND_LA_VALEUR ALGOBOX_RANDOM_INTEGER(1,100)
  essais PREND_LA_VALEUR 0
  proposition PREND_LA_VALEUR 0
  TANT_QUE (proposition != secret) FAIRE
    DEBUT_TANT_QUE
      LIRE proposition
      essais PREND_LA_VALEUR essais + 1
      SI (proposition < secret) ALORS
        DEBUT_SI
          AFFICHER "Trop petit !"
        FIN_SI
      SINON
        DEBUT_SINON
          SI (proposition > secret) ALORS
            DEBUT_SI
              AFFICHER "Trop grand !"
            FIN_SI
          SINON
            DEBUT_SINON
              AFFICHER "Bravo !"
            FIN_SINON
          FIN_SI
        FIN_SINON
      FIN_TANT_QUE
    FIN_TANT_QUE
  AFFICHER "Le nombre est : "
  AFFICHER secret
  AFFICHER " et vous avez mis "
  AFFICHER essais
  AFFICHER " essais pour le deviner."
FIN_ALGORITHME

```

```

    FIN_SI
  FIN_TANT_QUE
  AFFICHER "Trouve en "
  AFFICHER essais
  AFFICHER " essai(s)"
FIN_ALGORITHME

```

Correction 5.6 – Suite de Fibonacci

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  u0 EST_DU_TYPE NOMBRE
  u1 EST_DU_TYPE NOMBRE
  un EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  u0 PREND_LA_VALEUR 0
  u1 PREND_LA_VALEUR 1
  AFFICHER "u0 = "
  AFFICHER* u0
  SI (n >= 1) ALORS
    DEBUT_SI
      AFFICHER "u1 = "
      AFFICHER* u1
    FIN_SI
  POUR i ALLANT_DE 2 A n
    DEBUT_POUR
      un PREND_LA_VALEUR u0 + u1
      AFFICHER "u"
      AFFICHER i
      AFFICHER " = "
      AFFICHER* un
      u0 PREND_LA_VALEUR u1
      u1 PREND_LA_VALEUR un
    FIN_POUR
  FIN_ALGORITHME

```

Exemple : pour $n = 10$: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

Correction 5.7 – Plus grand commun diviseur (PGCD)

```

VARIABLES
  a EST_DU_TYPE NOMBRE
  b EST_DU_TYPE NOMBRE
  temp EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE a
  LIRE b
  TANT_QUE (b != 0) FAIRE
    DEBUT_TANT_QUE
      temp PREND_LA_VALEUR b
      b PREND_LA_VALEUR a MOD b
      a PREND_LA_VALEUR temp
    FIN_TANT_QUE
  AFFICHER "PGCD = "
  AFFICHER a

```

FIN_ALGORITHME**Trace pour PGCD(48, 18) :**

- $(a, b) = (48, 18) \rightarrow b \neq 0 \rightarrow (a, b) = (18, 48 \bmod 18) = (18, 12)$
- $(a, b) = (18, 12) \rightarrow b \neq 0 \rightarrow (a, b) = (12, 18 \bmod 12) = (12, 6)$
- $(a, b) = (12, 6) \rightarrow b \neq 0 \rightarrow (a, b) = (6, 12 \bmod 6) = (6, 0)$
- $b = 0 \rightarrow \mathbf{PGCD = 6}$

Correction 5.8 – Somme des chiffres

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  somme EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  somme PREND_LA_VALEUR 0
  TANT_QUE (n > 0) FAIRE
    DEBUT_TANT_QUE
      somme PREND_LA_VALEUR somme + (n MOD 10)
      n PREND_LA_VALEUR floor(n / 10)
    FIN_TANT_QUE
  AFFICHER "Somme des chiffres = "
  AFFICHER somme
FIN_ALGORITHME

```

Trace pour $n = 1234$:

- $n = 1234 \rightarrow \text{chiffre} = 4, n = 123, \text{somme} = 4$
- $n = 123 \rightarrow \text{chiffre} = 3, n = 12, \text{somme} = 7$
- $n = 12 \rightarrow \text{chiffre} = 2, n = 1, \text{somme} = 9$
- $n = 1 \rightarrow \text{chiffre} = 1, n = 0, \text{somme} = 10$
- $n = 0 \rightarrow \text{fin} \rightarrow \mathbf{\text{somme} = 10}$

Correction 5.9 – Nombre premier

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE
  estPremier EST_DU_TYPE BOOLEEN
DEBUT_ALGORITHME
  LIRE n
  estPremier PREND_LA_VALEUR VRAI
  SI (n < 2) ALORS
    DEBUT_SI
      estPremier PREND_LA_VALEUR FAUX
    FIN_SI
  i PREND_LA_VALEUR 2
  TANT_QUE (i <= floor(sqrt(n)) ET estPremier == VRAI) FAIRE
    DEBUT_TANT_QUE
      SI (n MOD i == 0) ALORS
        DEBUT_SI
          estPremier PREND_LA_VALEUR FAUX
        FIN_SI
      i PREND_LA_VALEUR i + 1
    FIN_TANT_QUE
  AFFICHER estPremier
FIN_ALGORITHME

```

```

    FIN_SI
    i PREND_LA_VALEUR i + 1
  FIN_TANT_QUE
SI (estPremier == VRAI) ALORS
  DEBUT_SI
    AFFICHER n
    AFFICHER " est premier"
  FIN_SI
SINON
  DEBUT_SINON
    AFFICHER n
    AFFICHER " n'est pas premier"
  FIN_SINON
FIN_ALGORITHME

```

Remarque

On teste les diviseurs de 2 à $\lfloor \sqrt{n} \rfloor$. Dès qu'on trouve un diviseur, on arrête (variable estPremier à FAUX).

6 Tableaux

Correction 6.1 – Remplissage et affichage d'un tableau

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  tab EST_DU_TYPE LISTE
  i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  POUR i ALLANT_DE 0 A n-1
    DEBUT_POUR
      LIRE tab[i]
    FIN_POUR
  AFFICHER "Tableau : "
  POUR i ALLANT_DE 0 A n-1
    DEBUT_POUR
      AFFICHER tab[i]
      AFFICHER " "
    FIN_POUR
  FIN_ALGORITHME

```

Correction 6.2 – Somme et moyenne d'un tableau

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  tab EST_DU_TYPE LISTE
  i EST_DU_TYPE NOMBRE
  somme EST_DU_TYPE NOMBRE
  moyenne EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  POUR i ALLANT_DE 0 A n-1

```

```

    DEBUT_POUR
        LIRE tab[i]
    FIN_POUR
    somme PREND_LA_VALEUR 0
    POUR i ALLANT_DE 0 A n-1
        DEBUT_POUR
            somme PREND_LA_VALEUR somme + tab[i]
        FIN_POUR
    moyenne PREND_LA_VALEUR somme / n
    AFFICHER "Somme = "
    AFFICHER somme
    AFFICHER " - Moyenne = "
    AFFICHER moyenne
FIN_ALGORITHME

```

Correction 6.3 – Recherche du maximum et du minimum

```

VARIABLES
    n EST_DU_TYPE NOMBRE
    tab EST_DU_TYPE LISTE
    i EST_DU_TYPE NOMBRE
    max EST_DU_TYPE NOMBRE
    min EST_DU_TYPE NOMBRE
    idxMax EST_DU_TYPE NOMBRE
    idxMin EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    POUR i ALLANT_DE 0 A n-1
        DEBUT_POUR
            LIRE tab[i]
        FIN_POUR
    max PREND_LA_VALEUR tab[0]
    min PREND_LA_VALEUR tab[0]
    idxMax PREND_LA_VALEUR 0
    idxMin PREND_LA_VALEUR 0
    POUR i ALLANT_DE 1 A n-1
        DEBUT_POUR
            SI (tab[i] > max) ALORS
                DEBUT_SI
                    max PREND_LA_VALEUR tab[i]
                    idxMax PREND_LA_VALEUR i
                FIN_SI
            SI (tab[i] < min) ALORS
                DEBUT_SI
                    min PREND_LA_VALEUR tab[i]
                    idxMin PREND_LA_VALEUR i
                FIN_SI
        FIN_POUR
    AFFICHER "Max = "
    AFFICHER max
    AFFICHER " (indice "
    AFFICHER idxMax
    AFFICHER ") - Min = "
    AFFICHER min
    AFFICHER " (indice "
    AFFICHER idxMin
    AFFICHER ")"
FIN_ALGORITHME

```

Correction 6.4 – Recherche d'une valeur

```
VARIABLES
  n EST_DU_TYPE NOMBRE
  tab EST_DU_TYPE LISTE
  i EST_DU_TYPE NOMBRE
  v EST_DU_TYPE NOMBRE
  trouve EST_DU_TYPE BOOLEEN
  nbOcc EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  POUR i ALLANT_DE 0 A n-1
    DEBUT_POUR
      LIRE tab[i]
    FIN_POUR
  LIRE v
  trouve PREND_LA_VALEUR FAUX
  nbOcc PREND_LA_VALEUR 0
  POUR i ALLANT_DE 0 A n-1
    DEBUT_POUR
      SI (tab[i] == v) ALORS
        DEBUT_SI
          AFFICHER "Valeur trouvee a l'indice "
          AFFICHER* i
          trouve PREND_LA_VALEUR VRAI
          nbOcc PREND_LA_VALEUR nbOcc + 1
        FIN_SI
      FIN_POUR
  SI (trouve == FAUX) ALORS
    DEBUT_SI
      AFFICHER "Valeur non trouvee"
    FIN_SI
  AFFICHER "Nombre d'occurrences : "
  AFFICHER nbOcc
FIN_ALGORITHME
```

Correction 6.5 – Tri par sélection

```
VARIABLES
  n EST_DU_TYPE NOMBRE
  tab EST_DU_TYPE LISTE
  i EST_DU_TYPE NOMBRE
  j EST_DU_TYPE NOMBRE
  idxMin EST_DU_TYPE NOMBRE
  temp EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  POUR i ALLANT_DE 0 A n-1
    DEBUT_POUR
      LIRE tab[i]
    FIN_POUR
  POUR i ALLANT_DE 0 A n-2
    DEBUT_POUR
      idxMin PREND_LA_VALEUR i
      POUR j ALLANT_DE i+1 A n-1
```

```

    DEBUT_POUR
        SI (tab[j] < tab[idxMin]) ALORS
            DEBUT_SI
                idxMin PREND_LA_VALEUR j
            FIN_SI
        FIN_POUR
        temp PREND_LA_VALEUR tab[i]
        tab[i] PREND_LA_VALEUR tab[idxMin]
        tab[idxMin] PREND_LA_VALEUR temp
    FIN_POUR
    AFFICHER "Tableau trie : "
    POUR i ALLANT_DE 0 A n-1
        DEBUT_POUR
            AFFICHER tab[i]
            AFFICHER " "
        FIN_POUR
    FIN_ALGORITHME

```

Remarque

La complexité du tri par sélection est $O(n^2)$ dans tous les cas. Il effectue $O(n)$ échanges.

Correction 6.6 – Tri à bulles

```

VARIABLES
    n EST_DU_TYPE NOMBRE
    tab EST_DU_TYPE LISTE
    i EST_DU_TYPE NOMBRE
    temp EST_DU_TYPE NOMBRE
    echange EST_DU_TYPE BOOLEEN
    passage EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    POUR i ALLANT_DE 0 A n-1
        DEBUT_POUR
            LIRE tab[i]
        FIN_POUR
    passage PREND_LA_VALEUR 0
    echange PREND_LA_VALEUR VRAI
    TANT_QUE (echange == VRAI) FAIRE
        DEBUT_TANT_QUE
            echange PREND_LA_VALEUR FAUX
            POUR i ALLANT_DE 0 A n-2
                DEBUT_POUR
                    SI (tab[i] > tab[i+1]) ALORS
                        DEBUT_SI
                            temp PREND_LA_VALEUR tab[i]
                            tab[i] PREND_LA_VALEUR tab[i+1]
                            tab[i+1] PREND_LA_VALEUR temp
                            echange PREND_LA_VALEUR VRAI
                        FIN_SI
                    FIN_POUR
                passage PREND_LA_VALEUR passage + 1
            FIN_TANT_QUE
        AFFICHER "Tableau trie : "
    POUR i ALLANT_DE 0 A n-1
        DEBUT_POUR

```

```

    AFFICHER tab[i]
    AFFICHER " "
  FIN_POUR
  AFFICHER " - Passages : "
  AFFICHER passage
FIN_ALGORITHME

```

Correction 6.7 – Comptage de fréquences

```

VARIABLES
  n EST_DU_TYPE NOMBRE
  notes EST_DU_TYPE LISTE
  freq EST_DU_TYPE LISTE
  i EST_DU_TYPE NOMBRE
  j EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  POUR i ALLANT_DE 0 A n-1
    DEBUT_POUR
      LIRE notes[i]
    FIN_POUR
  POUR i ALLANT_DE 0 A 20
    DEBUT_POUR
      freq[i] PREND_LA_VALEUR 0
    FIN_POUR
  POUR i ALLANT_DE 0 A n-1
    DEBUT_POUR
      j PREND_LA_VALEUR floor(notes[i])
      freq[j] PREND_LA_VALEUR freq[j] + 1
    FIN_POUR
  POUR i ALLANT_DE 0 A 20
    DEBUT_POUR
      AFFICHER "Note "
      AFFICHER i
      AFFICHER " : "
      POUR j ALLANT_DE 1 A freq[i]
        DEBUT_POUR
          AFFICHER "*"
        FIN_POUR
      AFFICHER* " "
    FIN_POUR
FIN_ALGORITHME

```

Remarque

On utilise `floor(notes[i])` pour convertir la note en entier, servant d'indice dans le tableau des fréquences. Chaque étoile représente une occurrence.

7 Fonctions

Correction 7.1 – Fonction carré

```

FONCTIONS_UTILISEES :
  FONCTION carre(x)
    DEBUT_FONCTION
      RETOURNER x * x
    FIN_FONCTION

VARIABLES
  x EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE x
  AFFICHER "Carre de "
  AFFICHER x
  AFFICHER " = "
  AFFICHER carre(x)
FIN_ALGORITHME
```

Correction 7.2 – Fonction estPair

```

FONCTIONS_UTILISEES :
  FONCTION estPair(n)
    DEBUT_FONCTION
      SI (n MOD 2 == 0) ALORS
        DEBUT_SI
          RETOURNER VRAI
        FIN_SI
      SINON
        DEBUT_SINON
          RETOURNER FAUX
        FIN_SINON
    FIN_FONCTION

VARIABLES
  n EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  POUR i ALLANT_DE 1 A 5
    DEBUT_POUR
      LIRE n
      SI (estPair(n) == VRAI) ALORS
        DEBUT_SI
          AFFICHER n
          AFFICHER " est pair"
        FIN_SI
      SINON
        DEBUT_SINON
          AFFICHER n
          AFFICHER " est impair"
        FIN_SINON
    FIN_POUR
  FIN_ALGORITHME
```

Correction 7.3 – Fonction maximum

```

FONCTIONS_UTILISEES :
  FONCTION max2(a, b)
    DEBUT_FONCTION
      SI (a > b) ALORS
        DEBUT_SI
          RETOURNER a
        FIN_SI
      SINON
        DEBUT_SINON
          RETOURNER b
        FIN_SINON
    FIN_FONCTION

VARIABLES
  a EST_DU_TYPE NOMBRE
  b EST_DU_TYPE NOMBRE
  c EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE a
  LIRE b
  LIRE c
  AFFICHER "Maximum = "
  AFFICHER max2(max2(a,b), c)
FIN_ALGORITHME

```

Remarque

On nomme la fonction max2 pour éviter le conflit avec un éventuel mot-clé. L'appel imbriqué max2(max2(a,b), c) calcule le maximum des trois.

Correction 7.4 – Fonction factorielle

Version itérative (AlgoBox ne supporte pas la récursivité) :

```

FONCTIONS_UTILISEES :
  FONCTION factorielle(n)
    DEBUT_FONCTION
      VARIABLES_LOCALES :
        i EST_DU_TYPE NOMBRE
        res EST_DU_TYPE NOMBRE
      res PREND_LA_VALEUR 1
      POUR i ALLANT_DE 1 A n
        DEBUT_POUR
          res PREND_LA_VALEUR res * i
        FIN_POUR
      RETOURNER res
    FIN_FONCTION

VARIABLES
  n EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE n
  AFFICHER n
  AFFICHER "! = "
  AFFICHER factorielle(n)
FIN_ALGORITHME

```

Remarque

La version récursive serait : si $n = 0$ retourner 1, sinon retourner $n \times \text{factorielle}(n - 1)$. AlgoBox ne la supportant pas, on utilise la boucle itérative.

Correction 7.5 – Fonction estPremier

```

FONCTIONS_UTILISEES :
  FONCTION estPremier(n)
    DEBUT_FONCTION
      VARIABLES_LOCALES :
        i EST_DU_TYPE NOMBRE
      SI (n < 2) ALORS
        DEBUT_SI
          RETOURNER FAUX
        FIN_SI
      i PREND_LA_VALEUR 2
      TANT_QUE (i*i <= n) FAIRE
        DEBUT_TANT_QUE
          SI (n MOD i == 0) ALORS
            DEBUT_SI
              RETOURNER FAUX
            FIN_SI
          i PREND_LA_VALEUR i + 1
        FIN_TANT_QUE
      RETOURNER VRAI
    FIN_FONCTION

VARIABLES
  a EST_DU_TYPE NOMBRE
  b EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE a
  LIRE b
  AFFICHER "Nombres premiers entre "
  AFFICHER a
  AFFICHER " et "
  AFFICHER b
  AFFICHER " : "
  POUR i ALLANT_DE a A b
    DEBUT_POUR
      SI (estPremier(i) == VRAI) ALORS
        DEBUT_SI
          AFFICHER " "
          AFFICHER i
        FIN_SI
      FIN_POUR
  FIN_ALGORITHME

```

Correction 7.6 – Fonction PGCD

```

FONCTIONS_UTILISEES :
  FONCTION pgcd(a, b)
    DEBUT_FONCTION

```

```

VARIABLES_LOCALES :
    temp EST_DU_TYPE NOMBRE
TANT_QUE (b != 0) FAIRE
    DEBUT_TANT_QUE
        temp PREND_LA_VALEUR b
        b PREND_LA_VALEUR a MOD b
        a PREND_LA_VALEUR temp
    FIN_TANT_QUE
    RETOURNER a
FIN_FONCTION

VARIABLES
    a EST_DU_TYPE NOMBRE
    b EST_DU_TYPE NOMBRE
    g EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE a
    LIRE b
    g PREND_LA_VALEUR pgcd(a, b)
    AFFICHER "Fraction simplifiée : "
    AFFICHER a/g
    AFFICHER "/"
    AFFICHER b/g
FIN_ALGORITHME

```

Exemple : $\frac{48}{18} \rightarrow \text{PGCD} = 6 \rightarrow \frac{8}{3}$

8 Exercices de synthèse

Correction 8.1 – Calculatrice simple

```

VARIABLES
    a EST_DU_TYPE NOMBRE
    b EST_DU_TYPE NOMBRE
    op EST_DU_TYPE CHAINE
    continuer EST_DU_TYPE CHAINE
    res EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    continuer PREND_LA_VALEUR "o"
    TANT_QUE (continuer == "o") FAIRE
        DEBUT_TANT_QUE
            LIRE a
            LIRE b
            LIRE op
            SI (op == "+") ALORS
                DEBUT_SI
                    res PREND_LA_VALEUR a + b
                FIN_SI
            SINON
                DEBUT_SINON
                    SI (op == "-") ALORS
                        DEBUT_SI
                            res PREND_LA_VALEUR a - b
                        FIN_SI
                    SINON
                        DEBUT_SINON

```

```

    SI (op == "*") ALORS
        DEBUT_SI
            res PREND_LA_VALEUR a * b
        FIN_SI
    SINON
        DEBUT_SINON
            SI (op == "/") ALORS
                DEBUT_SI
                    SI (b != 0) ALORS
                        DEBUT_SI
                            res PREND_LA_VALEUR a / b
                        FIN_SI
                    SINON
                        DEBUT_SINON
                            AFFICHER "Division par zero !"
                        FIN_SINON
                FIN_SI
            SINON
                DEBUT_SINON
                    AFFICHER "Operateur inconnu"
                FIN_SINON
            FIN_SI
        FIN_SINON
    FIN_SINON
    AFFICHER "Resultat = "
    AFFICHER res
    AFFICHER "Continuer ? (o/n) "
    LIRE continuer
    FIN_TANT_QUE
FIN_ALGORITHME

```

Remarque

On utilise une variable `continuer` de type CHAINE pour contrôler la boucle principale. L'utilisateur saisit "o" pour continuer ou toute autre valeur pour arrêter.

Correction 8.2 – Jeu du pendu simplifié

```

VARIABLES
    mot EST_DU_TYPE CHAINE
    affichage EST_DU_TYPE CHAINE
    lettre EST_DU_TYPE CHAINE
    essais EST_DU_TYPE NOMBRE
    trouve EST_DU_TYPE BOOLEEN
    i EST_DU_TYPE NOMBRE
    choix EST_DU_TYPE NOMBRE
    long EST_DU_TYPE NOMBRE
    nouvAff EST_DU_TYPE CHAINE
DEBUT_ALGORITHME
    choix PREND_LA_VALEUR ALGOBOX_RANDOM_INTEGER(1,5)
    SI (choix == 1) ALORS
        DEBUT_SI
            mot PREND_LA_VALEUR "PYTHON"
        FIN_SI
    SINON
        DEBUT_SINON

```

```

SI (choix == 2) ALORS
  DEBUT_SI
    mot PREND_LA_VALEUR "ALGOBOX"
  FIN_SI
SINON
  DEBUT_SINON
  SI (choix == 3) ALORS
    DEBUT_SI
      mot PREND_LA_VALEUR "FIBONACCI"
    FIN_SI
  SINON
    DEBUT_SINON
    SI (choix == 4) ALORS
      DEBUT_SI
        mot PREND_LA_VALEUR "EUCLIDE"
      FIN_SI
    SINON
      DEBUT_SINON
        mot PREND_LA_VALEUR "TABLEAU"
      FIN_SINON
    FIN_SI
  FIN_SINON
FIN_SINON
long PREND_LA_VALEUR mot.length
affichage PREND_LA_VALEUR ""
POUR i ALLANT_DE 0 A long-1
  DEBUT_POUR
    affichage PREND_LA_VALEUR affichage + "-"
  FIN_POUR
essais PREND_LA_VALEUR 0
TANT_QUE (essais < 8 ET affichage != mot) FAIRE
  DEBUT_TANT_QUE
    AFFICHER affichage
    LIRE lettre
    nouvAff PREND_LA_VALEUR ""
    POUR i ALLANT_DE 0 A long-1
      DEBUT_POUR
        SI (mot.substring(i,i+1) == lettre) ALORS
          DEBUT_SI
            nouvAff PREND_LA_VALEUR nouvAff + lettre
          FIN_SI
        SINON
          DEBUT_SINON
            nouvAff PREND_LA_VALEUR nouvAff
              + affichage.substring(i,i+1)
          FIN_SINON
        FIN_POUR
      affichage PREND_LA_VALEUR nouvAff
      essais PREND_LA_VALEUR essais + 1
    FIN_TANT_QUE
  SI (affichage == mot) ALORS
    DEBUT_SI
      AFFICHER "Gagne ! Le mot etait : "
      AFFICHER mot
    FIN_SI
  SINON
    DEBUT_SINON
      AFFICHER "Perdu ! Le mot etait : "

```

```

    AFFICHER mot
    FIN_SINON
FIN_ALGORITHME

```

Remarque

Cet exercice combine boucles, conditions et manipulations de chaînes. La logique de remplacement des tirets par les lettres trouvées est la partie la plus délicate.

Correction 8.3 – Statistiques de notes

```

VARIABLES
n EST_DU_TYPE NOMBRE
notes EST_DU_TYPE LISTE
i EST_DU_TYPE NOMBRE
somme EST_DU_TYPE NOMBRE
moy EST_DU_TYPE NOMBRE
variance EST_DU_TYPE NOMBRE
ecartype EST_DU_TYPE NOMBRE
auDessus EST_DU_TYPE NOMBRE
enDessous EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    somme PREND_LA_VALEUR 0
    POUR i ALLANT_DE 0 A n-1
        DEBUT_POUR
            LIRE notes[i]
            somme PREND_LA_VALEUR somme + notes[i]
        FIN_POUR
    moy PREND_LA_VALEUR somme / n
    AFFICHER "Moyenne = "
    AFFICHER moy
    variance PREND_LA_VALEUR 0
    auDessus PREND_LA_VALEUR 0
    enDessous PREND_LA_VALEUR 0
    POUR i ALLANT_DE 0 A n-1
        DEBUT_POUR
            variance PREND_LA_VALEUR variance
                + pow(notes[i] - moy, 2)
            SI (notes[i] >= moy) ALORS
                DEBUT_SI
                    auDessus PREND_LA_VALEUR auDessus + 1
                FIN_SI
            SINON
                DEBUT_SINON
                    enDessous PREND_LA_VALEUR enDessous + 1
                FIN_SINON
        FIN_POUR
    ecartype PREND_LA_VALEUR sqrt(variance / n)
    AFFICHER "Ecart-type = "
    AFFICHER ecartype
    AFFICHER "Au-dessus de la moyenne : "
    AFFICHER auDessus
    AFFICHER " - En dessous : "
    AFFICHER enDessous
    // Histogramme des mentions
    AFFICHER* "--- Mentions ---"

```

```

POUR i ALLANT_DE 0 A n-1
  DEBUT_POUR
    SI (notes[i] < 10) ALORS
      DEBUT_SI
        AFFICHER notes[i]
        AFFICHER " : insuffisant"
      FIN_SI
    SINON
      DEBUT_SINON
        SI (notes[i] < 12) ALORS
          DEBUT_SI
            AFFICHER notes[i]
            AFFICHER " : passable"
          FIN_SI
        SINON
          DEBUT_SINON
            SI (notes[i] < 14) ALORS
              DEBUT_SI
                AFFICHER notes[i]
                AFFICHER " : assez bien"
              FIN_SI
            SINON
              DEBUT_SINON
                SI (notes[i] < 16) ALORS
                  DEBUT_SI
                    AFFICHER notes[i]
                    AFFICHER " : bien"
                  FIN_SI
                SINON
                  DEBUT_SINON
                    AFFICHER notes[i]
                    AFFICHER " : tres bien"
                  FIN_SINON
                FIN_SI
              FIN_SINON
            FIN_SINON
          FIN_SINON
        FIN_POUR
      FIN_ALGORITHME

```

Correction 8.4 – Chiffrement de César

```

VARIABLES
  msg EST_DU_TYPE CHAINE
  cle EST_DU_TYPE NOMBRE
  i EST_DU_TYPE NOMBRE
  code EST_DU_TYPE NOMBRE
  chiffre EST_DU_TYPE CHAINE
  resultat EST_DU_TYPE CHAINE
  long EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE msg
  LIRE cle
  long PREND_LA_VALEUR msg.length
  resultat PREND_LA_VALEUR ""
  POUR i ALLANT_DE 0 A long-1
    DEBUT_POUR
      code PREND_LA_VALEUR caractereVersCode(

```

```

    msg.substring(i,i+1))
SI (code >= 65 ET code <= 90) ALORS
    DEBUT_SI
        code PREND_LA_VALEUR 65
        + (code - 65 + cle) MOD 26
    FIN_SI
SINON
    DEBUT_SINON
        SI (code >= 97 ET code <= 122) ALORS
            DEBUT_SI
                code PREND_LA_VALEUR 97
                + (code - 97 + cle) MOD 26
            FIN_SI
        FIN_SINON
    FIN_SINON
    chiffre PREND_LA_VALEUR codeVersCaractere(code)
    resultat PREND_LA_VALEUR resultat + chiffre
FIN_POUR
AFFICHER "Message chiffre : "
AFFICHER resultat
FIN_ALGORITHME

```

Remarque

On traite séparément les majuscules (codes 65–90) et les minuscules (codes 97–122). Les autres caractères (espaces, ponctuation) sont laissés inchangés. Le modulo 26 assure la rotation circulaire dans l'alphabet.

Correction 8.5 – Conversions numériques

Algorithme 1 : Décimal → Binaire

```

VARIABLES
    n EST_DU_TYPE NOMBRE
    binaire EST_DU_TYPE CHAINE
    reste EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
    LIRE n
    binaire PREND_LA_VALEUR ""
    SI (n == 0) ALORS
        DEBUT_SI
            binaire PREND_LA_VALEUR "0"
        FIN_SI
    SINON
        DEBUT_SINON
            TANT_QUE (n > 0) FAIRE
                DEBUT_TANT_QUE
                    reste PREND_LA_VALEUR n MOD 2
                    SI (reste == 0) ALORS
                        DEBUT_SI
                            binaire PREND_LA_VALEUR "0" + binaire
                        FIN_SI
                    SINON
                        DEBUT_SINON
                            binaire PREND_LA_VALEUR "1" + binaire
                        FIN_SINON
                    n PREND_LA_VALEUR floor(n / 2)
                FIN_TANT_QUE
            FIN_SINON
        FIN_SINON
    FIN_TANT_QUE

```

```

    FIN_SINON
    AFFICHER "Binaire : "
    AFFICHER binaire
FIN_ALGORITHME

```

Trace pour $n = 13$:

- $13 \div 2 = 6$ reste 1 $\rightarrow$ binaire = "1"
- $6 \div 2 = 3$ reste 0 $\rightarrow$ binaire = "01"
- $3 \div 2 = 1$ reste 1 $\rightarrow$ binaire = "101"
- $1 \div 2 = 0$ reste 1 $\rightarrow$ binaire = "1101"

Résultat : $13_{10} = 1101_2$

Algorithme 2 : Binaire $\rightarrow$ Décimal

```

VARIABLES
    binaire EST_DU_TYPE CHAINE
    long EST_DU_TYPE NOMBRE
    i EST_DU_TYPE NOMBRE
    decimal EST_DU_TYPE NOMBRE
    bit EST_DU_TYPE CHAINE
DEBUT_ALGORITHME
    LIRE binaire
    long PREND_LA_VALEUR binaire.length
    decimal PREND_LA_VALEUR 0
    POUR i ALLANT_DE 0 A long-1
        DEBUT_POUR
            bit PREND_LA_VALEUR binaire.substring(i,i+1)
            SI (bit == "1") ALORS
                DEBUT_SI
                    decimal PREND_LA_VALEUR decimal
                        + pow(2, long-1-i)
                FIN_SI
            FIN_POUR
        AFFICHER "Decimal : "
        AFFICHER decimal
    FIN_ALGORITHME

```

Trace pour "1101" :

- bit '1' (position 0) : $1 \times 2^3 = 8$
- bit '1' (position 1) : $1 \times 2^2 = 4$
- bit '0' (position 2) : $0 \times 2^1 = 0$
- bit '1' (position 3) : $1 \times 2^0 = 1$
- Total : $8 + 4 + 0 + 1 = 13$

Résultat : $1101_2 = 13_{10}$